

Bash / shell SCRIPT

SCRIPT ← عبارة عن **File** حواه مجموعة من الأوامر التي دأبنا بنفذها وبدل ما كل مرة أنفذ الأوامر أقد من قلال ال **SCRIPT** أنى أنفذها على طول ويشكل **Automation** من قلال ال **SCRIPT**

ال **SCRIPT** يحل مشاكل أي

1] نسبة الخطأ 90%

2] افتتار الوقت فلا من بقت تتكامل لومدها

3] توفير ال **resources** هي من متاع محاي العنصر البشري بدل ما يكون مشغول بمجرد ان يعمل **BooK** ~~في~~ فلا من ال **SCRIPT** هو ال **Automation**

طريقة العمل

عبارة عن **interpreter** ودا بيأخذ لسلار سطر ويبدا بنفذ ال **SCRIPT** لوفى على على في ال **SCRIPT** هيظهر قبل ما الكود يخلص وساعتها الكود يوقف

الحاجات الأساسية المطلوبة سواء كنت **Dev** أو **Ops**

1] **Python script** 2] **Bash script**

3] **Automation tool** ← **Ansible**

ال **DevOps** عبارة عن **Integration [CI]** و **Automation [CD]**

من اهم الحاجات اللي يتميز **DevOps** عن الثاني انه واصل لحد في في شركتك على ال **Automation**

Write Script

جعل ملف ونكتب فيه ال script ونمنع للملف
ال execute Permission

- [1] Create File
- [2] edit/add Code
- [3] Save
- [4] Change Permission X

- [5] run/execute script
- ./script
 - Full Path/
 - sh script or Path
 - Bash script or Path
 - Source script or Path

حاجات مهمة لكتابة ال script

[1] Mandatory

[1] لازم الأسكريبت يبدأ بـ `#!/Path/to/Bash` ويختلف ال Path

على حسب لغة الكود لو `Python` بحد ال Path بقاع ال `Python`

!# Shebang عبارة عن label بيبدأ بي اول

سطر في ال script اللي يعرف ال System ان دا script

[2] بيبدأ في ال Shell اللي بيستخدمها

[2] Optional

Clean Code

[1] الكود يكون منظم و Readable

ال Clean Code بيبدأ باسم الملف

Bash/Shell Script

يتعمل ال Shell ويتعرف على موجهة ولاعن طريق
أمر Which bash يظهر لك ال Path بتاعها

دائما اسم ال script اخرها sh. يستحسن يكون اخرها sh
عشان هو Shell

ليه يستحسن يكون اخرها
عشان اقدر اقول ال scripts لازم يكون ليها extension او حاجة
مميزة ليها

مش شرط عشان يكون script يبقى اخره sh.

كتابة script لملأه Vim Welcome.sh

```
#!/usr/bin/bash
#this script Print Welcom
echo "Welcome to DevOps"
date
```

بعد الكتابة وتعمل Save وتخرج هتدي للملف Permission X
Chmod u+x
دلوقة اقدر اعمل run

Vim Print info

info user - Who user? - Location Your? - id user - date

Whoami

Pwd

id -u

date +%F

ls --color

عشان تظهر الألوان

بيظهر التاريخ بشكل عادي

في ال LS لما يكون في file

ممكن ملونة لازم نكتب

free -m ← بيظهر معلومات عن الذاكرة

Cat /Proc/CPUinfo | grep Processor ← عدال CPU

~~uptime~~ ← بيظهر ال System load

Load average : 0.31, 0.07, 0.02

أفركا دقة / أفركا دقايق / معنى القيمة دي ؟

الثلاث متاجين تحرق في

عندي كالت CPU مثلاً 3

يعني القيمة دي كل 1

بيستهلك CPU كامل بس

بيتوزع على ال 3 مش بيكون

العمل على CPU بس

* # الأرقام دي ال Average

Load average ← مجموعة ال Tasks اللي شغالة بتستهلك
قد أفركا



Disk uses info

df -h disk ~~ال~~ بيظهر معلومات عن ال

لازم وفنروري اجابا يكون ال Code ReaderPal

Shell Feature

11 And (\$&)

```
#!/usr/bin/bash
ls --color /homeeee
echo "Welcome DevOps"
```

Error

فيما عايناه
من قبل

يحدث

هنا في And عملت Links بين الأوامر حيث تستغل غير لما
الأمر الأول يستغل الأول

ls \$& chmod u+x

12 OR (||)

هنا في تنفيذ الأول والثاني لو الأول لم ينجح عادي هيدخل
ينفذ الأمر الثاني

ls || chmod u+x

13 cmd sub \$(cmd)

لو انا في متاي اخرين محتاج ادمجهم مع بعض عشان ينفذولي
task واحدة

touch file1 - \$(date +%F)

بستخدمها في حالة آي ؟

لو انا عملت script بي عمل Backup وانا غير لما يعمل Backup
يضيف تاريخه اسم ال file ليحل التاريخ

touch file1 - \$(date +%F) - \$(Whoami).txt

كل output هيكون هو اسم ال file خليه كل حاجة
كانها امر واحد من غير ما يكون الأمر independent

Independent كل امر في سطر مستقل
الأمر كأنه امر واحد

And + OR

Substitute \$

Shell Features

Redirection & Pipe

(>) (|)

Pipe (|) → يأخذ الناتج من خطوة في عملية ثانية
 Redirection (>) → يأخذ الناتج ويخزنه

Pipe

df -h | grep dev

هنا @ هي في الأمر df -h وهي تغل عملية ثانية فيه

df -h | cut -f1 -d" " | sort | grep tmpfs

Redirection

df -h > file.txt

تخزن ال output في file

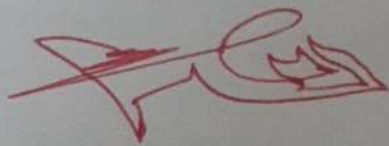
df -h | cut -f1 -d" " | sort | grep tmpfs > file

أقصد Redirect مع ال Pipe

بمعنى كذا هي عمل overwrite على المحتوى الذي في الملف

ساعتها عمل Append >> file

df -h | cut -f1 -d" " | sort >> file



Shell Features

Redirection

Redirect

١١ بتخزن النتيجة لكن ميفرنش ال ~~Error~~ سواء كان > أو >>

١٢ لو انت عايز تخزن الميع والغلط أو تخزن كل حاجة في مكان

١٣ عايز افزن كله في نفس الملف سواء الغلطا أو الميع
ls /home /usr &> all.txt

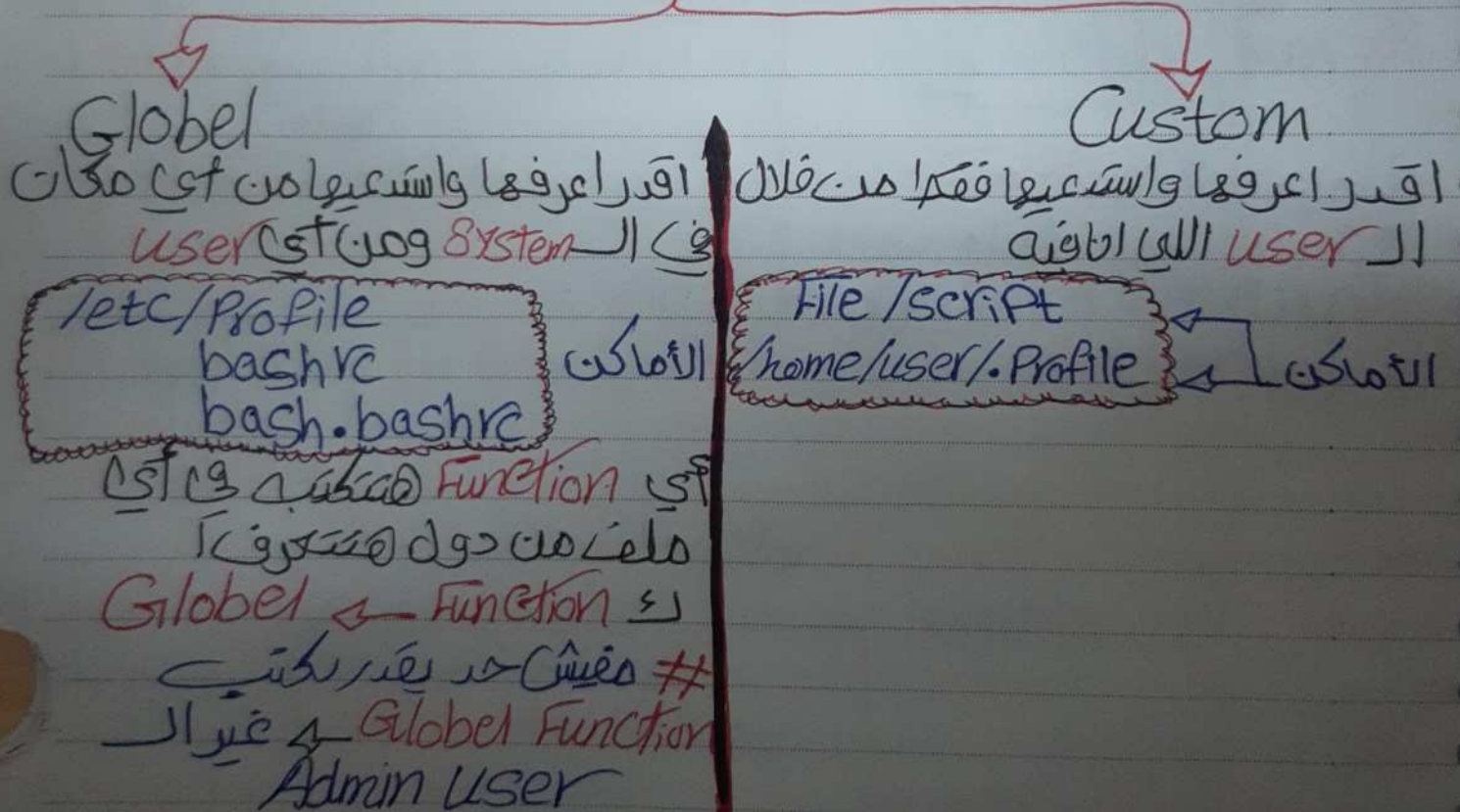
١٤ عايز افزن كل حاجة في مكان مختلف
ls /home /usr >> out1.txt >> out2.txt

الميزة هنا لو عندي script كبر حجمه أقدر احط
أي غلط في ملف جديد لما يطول غلطه أقدر اعالج الغلطا
وأعمل عليها وكله يبقى تمام

Function & variables

Function : لو عندك مجموعة من الأوامر بتكرر انامش عايز اكرها كل اللوحة لو عندك امر بتكرر 20 مرة مثلاً مش منطقي تكتبه 20 مرة في الكود بتاعك ساعتها تعمل **Function** وتستعملها لما تكتب تنفذ الأوامر

Function



```

Name() {
    Tasks
}
    
```

لي انا عرفت
function

Functions

Vim function.sh ①

②

```
#!/usr/bin/bash
Print () {
    echo "Welcome to Linux"
    date +%F
    Pwd
    F
}
Print ← كذا أنا اسمي في الدالة
ls --Color
free -m
Print
```

③ chmod +x function.sh

④ ./function.sh

Sudo vim /etc/profile

Global Fun #

تعريف الدالة Function
في ملف من الملفات
الثلاثة التي قلنا عليه
يعرف في user

fun # التعريف عن كتابة السؤال

Print

لو حبيت التعريف في Function من ملفات
صغيرة واكتب

```
#!/usr/bin/bash
source /etc/profile
Print ←
```

كذا أنا اسمي في
الدالة Source واسمي
Print من الدالة Source
وال Source الاسمي الاوامر من

/etc/profile

Variables & Alias

Variables ← هو مكان يخزن فيه قيمة متغيرة واسمها

وقت الفياجي

دیک

load = \$(uptime | awk '{print \$8}' | cut -f1 -d ",")
echo \$load ← خروجی از دستور

→ جميع الامور

② जोषीर

Read -p "Enter Your name" name:

المخبر الذي همموا بقتل فيه
القسمه

الحمد لله

செயல்பாடு - படிப்பது

```
echo "Enter your name"
```

echo \$name	read name
-------------	-----------

② age = 10

echo \$age

١) اعراف Var وظيفية ياف ال Variables بكلمة من امر تاني

بالتالي يكون

2] اعراف Variable و Read

(3) اعراف Variables و Static

Variables

Global

System / env

دائما ال Var

System الخاصة بـ

Capital حروفه

يتعرف ال System Var

متعلقه ال

echo \$VAR ← Capital

/etc/Profile #

~~ba~~ bashrc #

bash-bashrc #

* env

ال Var يظهر كل ال
الخاصة بال System

Custom

الفضل ال Var

الخاصة بـ user

تكون Small

لأن كل ال Var

الخاصة بـ System

تكون Capital

بحيث هي محال

تعارف

يتعرفها

Small ← echo \$var

~~/home/user/.Profile~~ #

/home/user/.bashrc

* Set

usr Print ال Vars

globe ال Func

usr

globe

ال ال Print كبير جدا واضعنا

ال Var الخاصة بـ user

Set | grep '[a-z]' | grep -v '(' | grep '='

الظهر لي كل حروفه من a-z

الظهر لي كل حروفه
فيهها اقواس لأن

الاقواس اللي بتدور ال fun

الظهر الحاجة اللي فيها =

Alias

[Shortcut Command] دھمتہ بیخبر آئے امر بدل ما آکتبہ امر کبیر گل لکویہ

alias userVar = 'set | grep '[a-z]' | grep -v "(" | grep "="'
userVar واسطیہ یکل بسلاطہ آکتبہ

ہیتر وضع الامر داغے

Global

/etc/profile

bashrc

bash.bashrc

Custom

هدفہا انی افنمر اوامر کتیر مع بعضی بامر و امر
ذی ماتقول کدا کائناتک بدضرع امر

Fun / Var / Alias

Alias → الأسهل في الاختصار والتأخير

Var → لوفي قيمة مخزنة واستدعيها

Fun → لما يكون عندي امر يحتاج Input من User وعلى أساسها ينفذ Input ونحصل عليه على شكل

كل Features تقرر تعمل بشكل الثانية
بشكل كل واحدة مستخدمة في حاجة

Condition & loop

loop ← لو انا عندي **Function** هسرعها كل لشوية
عشان اقترانقها طيب لو انا محتاج انفذ ال **Function** مثلاً
50 مرة هل مني كل لشوية اسرعها ل **50 مرة** أكيد لا
~~وظيفة~~ ال **loop** ← مثلاً عندي **Task** وعاليز اكرها 100 مرة

Condition ← مثلاً عاليز اعمل **run** لسكريبت معين لعاليز يكون
بشكله أو جزء من ال **Script** يشغل وجزء لا وهنا فكرة
ال **Condition**

Condition

Single Condition (if)

```
if [ condition ]
then
    Tasks
else
    Tasks
fi
```

القالب بناسه ال if

multi Condition
(if + elif)

```
if [ condition ]
then
    tasks
elif [ condition ]
then
    tasks
else
    tasks
fi
```

Operators

Operators

equal $\rightarrow (=)$ OR $(-eq)$
 String int

not equal $\rightarrow (!=)$ OR $(-ne)$

greater than $\rightarrow (>)$ OR $(-gt)$

greater OR equal $\rightarrow (>=)$ OR $(-ge)$

less than $\rightarrow (<)$ OR $(-lt)$

less than OR equal $\rightarrow (<=)$ OR $(-le)$

/ /

Queso Rango de Suizo
Suizo

↑
تغير
Var ()

↑
تغير
Value
وغير متغير
Var ()

TASKS + \$Var

★ Exercises
Variation

~~Qatar~~

Condition
Fates (بی)

مثال: لو عايز افعل Task تمشي
لحد الساعة 4 ولما الساعة توصل 4
ال Task توقف ساعتها ال While
تمشك

بين ال while خريطة ال Condition
آل ال until خريطة ال Condition

done